

Modèle de données BDOH

Points ouverts et questions de conception

Louis Héraut · mai 2026

INRAE, UR RiverLy, Villeurbanne, France

TABLE DES MATIÈRES

À quoi sert ce document

Partie A - Chantiers structurels

- A1. La couche Transformation comme moteur d'exécution
- A2. Incertitude de mesure
- A3. Enrichissement de Bundle pour la publication et l'interopérabilité
- A4. Historisation des jeux de fonctions de transfert et versionnement des séries dérivées

Partie B - Décisions en attente

- B1. La frontière enum SQL / vocabulaire Keyword

Partie C - Ambiguïtés locales et points à vérifier

- C1. ControlObservation : ancrage et qualité
- C2. Procédure optionnelle sur Datastream vs obligatoire sur TimeSeries
- C3. Asymétrie KeywordAssignment / KeywordRequirement
- C4. Specimen.operator : pertinence de la valeur `Machine`

Partie D - Veille sur les standards externes

Tableau de synthèse

À quoi sert ce document



Ce document recense ce qui **n'est pas encore tranché** dans le modèle de données BDOH : ambiguïtés de conception, décisions en attente, et chantiers structurels identifiés mais non spécifiés.

Il ne décrit pas le modèle (cela, c'est `modele_donnees_v12.md`). Il décrit les endroits où le modèle attend encore une décision.

Chaque point est présenté de la même façon : l'état actuel, la question posée, et, quand c'est pertinent, les pistes envisagées. L'objectif est que chaque collègue puisse réagir, compléter, ou trancher un point en connaissance de cause.

Comment lire les niveaux de priorité :

- **Structurel** - conditionne plusieurs décisions en aval ; à traiter avant d'avancer beaucoup plus loin.
- **Important** - affecte la robustesse ou l'interopérabilité, mais n'empêche pas d'avancer ailleurs.
- **À clarifier** - ambiguïté locale ou question de documentation ; faible risque, mais mérite d'être tranché pour la propreté du modèle.

Partie A - Chantiers structurels



Ces quatre points sont les plus importants. Chacun est un sujet de conception à part entière, pas une simple correction.

A1. La couche Transformation comme moteur d'exécution



Priorité : structurel

État actuel. Le modèle a une couche transformation centrée sur les fonctions de transfert : `TransferFunction` , `TransferFunctionPoint` , `TransferFunctionSet` , `TransferFunctionBatch` , et `TransformationBatch` qui applique un `transferFunctionSet` (champ obligatoire) pour produire une `TransformedTimeSeries` .

Ce qui a été clarifié. BDOH *exécute* les transformations (il ne se contente pas de les documenter). Les calculs seront réalisés par des runners, qui seront représentés comme des entités `Machine` . Le cas implémenté aujourd'hui - les fonctions de transfert, type courbe de tarage / barème - n'est qu'un cas particulier.

Le problème. `TransformationBatch.transferFunctionSet` est obligatoire. Cela force toute transformation à passer par un jeu de fonctions de transfert. Or il existe d'autres familles de transformations qui n'ont rien d'une courbe de tarage :

- agrégations temporelles (par exemple le débit journalier maximal annuel) ;
- comblement de lacunes ;
- corrections (dérive de capteur, correction chimique ad hoc) ;
- combinaisons de plusieurs séries d'entrée.

Questions ouvertes.

1. Comment représenter une transformation qui n'est pas une fonction de transfert ? Faut-il rendre `transferFunctionSet` optionnel et ajouter un mécanisme alternatif (référence à un algorithme, paramètres d'exécution) ?
2. Où documenter l'algorithme exécuté et ses paramètres ? Sur la `Machine` (le runner) ? Sur le `TransformationBatch` ? Sur la `Procedure` (type=transformation existe déjà) ?
3. Faut-il un objet commun « transformation » qui unifie tous les cas, avec les fonctions de transfert comme une spécialisation parmi d'autres ?
4. Les paramètres d'exécution d'un runner doivent-ils être stockés (pour la reproductibilité) ou seulement référencés ?

Piste évoquée. Un `TransformationBatch` avec `transferFunctionSet` en `0..1` et un second mécanisme `0..1` pour les autres cas (référence algorithme + paramètres). La forme exacte reste à définir - c'est le cœur du chantier.

A2. Incertitude de mesure



Priorité : structurel

État actuel. Le seul champ d'incertitude du modèle est `TransferFunctionPoint.uncertainty`. Les observations elles-mêmes (`Observation` , `ValidatedObservation` , `Transformation`) ne portent aucune incertitude.

Le problème. Pour des données qui revendiquent une reproductibilité scientifique, l'absence d'incertitude sur les valeurs mesurées est une lacune réelle. Les standards de référence la portent : ODM2 a un `resultUncertainty` , Helmholtz SMS gère l'incertitude au niveau des observations.

Questions ouvertes.

1. À quel niveau attacher l'incertitude : sur chaque observation ? sur la série (incertitude type homogène) ? les deux ?
2. Quel modèle d'incertitude : valeur unique ($\pm x$) ? intervalle de confiance ? référence à une méthode d'estimation ?
3. L'incertitude est-elle saisie, importée du capteur, ou calculée lors de la validation / transformation ?
4. Comment se propage-t-elle à travers une transformation (une `TransformedTimeSeries` hérite-t-elle d'une incertitude calculée à partir de celle de ses entrées) ?

A3. Enrichissement de Bundle pour la publication et l'interopérabilité



Priorité : structurel

État actuel. `Bundle` est défini comme objet de publication (regroupement de séries, fonctions, observations de contrôle). Il porte `code` , `name` , `license` . Il n'a aucun champ de métadonnée de publication citable.

Le problème. Pour faire un vrai pont avec les entrepôts de données de recherche - Recherche Data Gouv, Theia/OZCAR, et l'ENVRI-Hub au niveau européen - Bundle a besoin des métadonnées attendues par ces plateformes : attribution, date de publication, version, identifiants liés, etc.

Questions ouvertes.

1. Quels champs DataCite ajouter : `publisher` , `publicationYear` , `version` , `relatedIdentifier` , `resourceType` ?
2. Le DOI d'un Bundle est-il porté par `Identifier` (codeType= `doi`) ou par un champ dédié ?
3. Comment gérer le versionnage d'un Bundle publié (un Bundle v2 qui corrige un Bundle v1 déjà cité) ?
4. Quelle correspondance exacte avec DCAT pour l'export catalogue ? Un Bundle est-il un `dcat:Dataset` , une `dcat:Distribution` , ou les deux selon le contexte ?
5. Faut-il distinguer un Bundle « brouillon » d'un Bundle « publié et figé » ?

A4. Historisation des jeux de fonctions de transfert et versionnement des séries dérivées



Priorité : structurel

État actuel. Un `TransferFunctionSet` est la composition temporelle des `TransferFunction` actives sur une station (la succession des courbes de tarage). Une `TransformedTimeSeries` est calculée en appliquant ce TFSet. Quand une nouvelle courbe de tarage est établie, la composition du TFSet est modifiée, et les `TransformedTimeSeries` qui en dépendent sont recalculées.

Le problème. Le recalcul écrase les valeurs précédentes. Or les débits produits par l'ancienne version d'un TFSet ont pu être consultés, cités ou publiés par des tiers. Les faire disparaître silencieusement casse la reproductibilité scientifique : une étude qui s'appuyait sur les anciens débits ne peut plus être refaite. Une donnée publiée ne doit pas disparaître parce que la méthode de calcul a évolué.

Deux niveaux de réponse, à distinguer.

1. *Historiser le TFSet* : conserver la trace des compositions successives, à quelle période le TFSet était {courbe A, B}, à quelle période il est devenu {courbe A, B, C}. C'est l'historique de la méthode de calcul.
2. *Versionner la TransformedTimeSeries* : conserver les résultats eux-mêmes, la série de débits calculée avant le changement reste accessible à côté de la nouvelle, plutôt que d'être écrasée.

Ces deux niveaux ne sont pas équivalents. Si le TFSet est historisé et que BDOH sait rejouer un calcul (cf. A1, BDOH exécute les transformations), alors l'ancienne série peut en principe être recalculée à la demande sans être stockée. Versionner la TTS, à l'inverse, garantit un accès immédiat à l'ancienne série mais a un coût de stockage et de gestion.

Questions ouvertes.

1. Historiser le TFSet : par le pattern d'association datée (un TFSet est déjà une composition datée de `TransferFunction`) ou par un mécanisme de versionnement explicite ?
2. Faut-il stocker les anciennes `TransformedTimeSeries`, ou suffit-il de pouvoir les recalculer à partir de l'historique du TFSet et de la procédure (lien direct avec A1) ?
3. Le versionnement d'une TTS est-il systématique, ou laissé au choix des curateurs (« garder cette version-ci parce qu'elle a servi ») ?
4. Comment un utilisateur cite-t-il une version précise d'une série dérivée, pour que sa publication reste reproductible ? Lien avec le DOI et le versionnement de Bundle (cf. A3).
5. Quelle articulation avec `qualityFlag` et le statut : une ancienne version est-elle marquée « dépréciée mais conservée » ?

Lien avec les autres points. Ce chantier dépend de A1 (la capacité de rejouer un calcul change la réponse à la question 2) et recoupe A3 (citer une version précise d'une série rejoint le versionnement de Bundle).

Partie B - Décisions en attente



Points plus circonscrits que la partie A, mais qui demandent une décision explicite avant d'être considérés comme réglés.

B1. La frontière enum SQL / vocabulaire Keyword



Priorité : important

État actuel. Le modèle a deux mécanismes pour les valeurs contrôlées :

- des **enums SQL** figés dans le schéma : `status` , `qualityFlag` , `systemType` , `agentType` , `anchorType` , `resourceType` , `codeType` , `Procedure.type` , `origin` , `acquisitionType` , `aggregationStatistic` , `TransferFunctionSet.type` , `depthReference` , `validationMode` , `transmissionMode` ;
- le **quadriptyque Keyword** (`KeywordType` / `Keyword` / `KeywordAssignment` / `KeywordRequirement`) pour les vocabulaires évolutifs gérés sans migration.

Le critère actuel est : enum SQL si la valeur conditionne du code applicatif, Keyword sinon.

Le problème. Le critère est sain mais la frontière reste floue. Certains enums sont de vrais discriminants techniques (`agentType` , `anchorType` , `qualityFlag`) - ils doivent rester en SQL. Mais d'autres sont des vocabulaires métier qu'on a figés par confort :

- `aggregationStatistic` est aligné sur ODM2 - si ODM2 évolue, c'est une migration SQL.
- `Procedure.type` est une classification métier (sampling | observation | modeling | aggregation | transformation | validation).
- `systemType` est à la fois un discriminant TPC et une classification.

Questions ouvertes.

1. Pour chaque enum « vocabulaire métier » (`aggregationStatistic` , `Procedure.type` ...), confirme-t-on le choix de la rigidité SQL, ou bascule-t-on vers Keyword ?
2. Peut-on formuler un critère plus net que « conditionne du code » - par exemple : « discriminant de structure » (reste SQL) vs « classification de contenu » (peut devenir Keyword) ?
3. Quels enums actuels sont les plus susceptibles d'évoluer dans le temps, et donc les plus à risque s'ils restent en SQL ?

Ce point n'est pas bloquant aujourd'hui - le modèle fonctionne. Mais il mérite d'être tranché consciemment, enum par enum, plutôt que laissé implicite.

Partie C - Ambiguïtés locales et points à vérifier



Points de faible risque, signalés pour ne pas les oublier. La plupart se règlent par une décision courte ou une ligne de documentation.

C1. ControlObservation : ancrage et qualité



Priorité : à clarifier

Deux interrogations sur `ControlObservation` :

1. **Pas de `qualityFlag`** . Une observation de contrôle (jaugage de vérification, mesure de référence) peut elle-même être de qualité douteuse. Comment le signale-t-on ? Faut-il un `qualityFlag` sur `ControlObservation` ?
2. **Lien vers `System sensor`**. `ControlObservation` référence un `System` de type `sensor`. Or les autres entités de mesure (`TimeSeries`, `Datastream`) se rattachent à un contexte géographique via le pattern anchor, et passent par un `Deployment` pour la dimension instrumentale. `ControlObservation` semble court-circuiter ce schéma. Est-ce volontaire (une observation de contrôle est ponctuelle et n'a pas de déploiement durable) ou est-ce une incohérence à corriger ?

C2. Procédure optionnelle sur `Datastream` vs obligatoire sur `TimeSeries`



Priorité : à clarifier

`Datastream.procedure` est `0..1` (optionnel), alors que `TimeSeries.procedureObservation` est `1` (obligatoire).

C'est probablement volontaire : un flux brut doit pouvoir être déposé dès l'installation, même si le protocole de mesure n'est pas encore renseigné - il sera enrichi plus tard. La série métier validée, elle, exige un protocole défini. Si cette lecture est correcte, il suffit de l'expliciter dans une note ; sinon, c'est une asymétrie à corriger.

C3. Asymétrie `KeywordAssignment` / `KeywordRequirement`



Priorité : à clarifier

`KeywordAssignment.resourceType` admet 15 valeurs (dont `Datastream`, `Deployment`, `Bundle`). `KeywordRequirement.resourceType` en admet 11 (sans ces trois).

C'est cohérent en soi : on peut *assigner* un mot-clé à un `Datastream` sans qu'il existe pour autant une *règle de complétion obligatoire* sur les `Datastreams`. Les deux domaines n'ont pas de raison d'être identiques. Le point est simplement signalé pour que cette asymétrie soit validée consciemment, et non perçue plus tard comme un oubli.

C4. Specimen.operator : pertinence de la valeur **Machine**



Priorité : à clarifier

Specimen.operator utilise le pattern TPC agent, donc admet **Person** ou **Machine**. Un prélèvement d'échantillon est le plus souvent fait par une personne. La valeur **Machine** a toutefois été conservée : des préleveurs automatiques existent et ce cas commence à se présenter. Point considéré comme réglé (conservation de **Machine**), signalé seulement pour mémoire.

Partie D - Veille sur les standards externes



Le modèle s'aligne sur des standards qui ont évolué récemment. Ces évolutions ne demandent pas d'action immédiate mais doivent être suivies, en particulier pour une éventuelle v2.

- **OGC API - Connected Systems (Parts 1 & 2)** - approuvés comme standards OGC officiels (annonce début 2026). Successeur de STA, SOS et SPS. CS API est conçu pour coexister avec STA, pas pour le remplacer brutalement. La refonte instrumentale du modèle (System unifié + Deployment récursif) est déjà alignée sur ce standard. À surveiller pour une cible v2.
- **OGC SensorThings API 2.0** - en cours de ratification (période de commentaires publics close en janvier 2026). À suivre, notamment pour les évolutions touchant `unitOfMeasurement` et la structure des Datastreams.
- **STAMPLATE (Helmholtz)** - le profil de métadonnées STA pour l'environnement a livré son schéma formel (dépôt Zenodo, 2025). C'est la référence concrète pour structurer les champs `properties` exposés en STA.
- **eLTER-RI / ENVRI-Hub NEXT** - l'interopérabilité européenne des infrastructures de recherche environnementale converge vers DCAT-AP et ISO 19115. Pertinent pour la stratégie d'export catalogue de Bundle (cf. A3).

Tableau de synthèse



#	POINT	PRIORITÉ	TYPE DE RÉOLUTION ATTENDUE
A1	Transformation comme moteur d'exécution	Structurel	Conception d'un sous-modèle
A2	Incertitude de mesure	Structurel	Conception d'un sous-modèle
A3	Enrichissement Bundle / publication	Structurel	Ajout de champs + alignement
A4	Historisation TFSet / versionnement des TTS	Structurel	Conception d'un sous-modèle
B1	Frontière enum SQL / Keyword	Important	Décision enum par enum
C1	ControlObservation : ancrage et qualité	À clarifier	Décision + éventuelle correction
C2	Procédure optionnelle DS vs obligatoire TS	À clarifier	Note de documentation
C3	Asymétrie KeywordAssignment / Requirement	À clarifier	Validation consciente
C4	Specimen.operator : valeur Machine	À clarifier	Réglé, pour mémoire
D	Veille standards externes	-	Suivi continu